TAMPER-EVIDENT LOGGING SYSTEM

Technical Design Report & Implementation Documentation

Subject	Secure Audit Logging & Cryptographic Integrity
Language	Python 3.x
Algorithm	SHA-256 Blockchain-style Hash Chain
Storage	Persistent JSON File
Date	31 March 2026

Submitted by - Chandrashekhar Tyagi
chandershekhar8433@gmail.com

1. Introduction

In modern computing environments, audit logs serve as the authoritative record of system activity. They are essential for security investigations, compliance requirements, and forensic analysis. However, a log is only trustworthy if it cannot be secretly modified — if an attacker or insider can alter, delete, or reorder entries without detection, the entire audit trail loses its evidentiary value.

This report documents the design and implementation of a Tamper-Evident Logging System — a Python-based solution that employs cryptographic hash chaining to ensure that any unauthorized modification of log data is immediately detectable. The system mirrors the fundamental integrity mechanism used in blockchain technology and secure audit systems found in financial, healthcare, and government applications.

1.1 Problem Statement

Traditional log files stored as plain text or simple databases are vulnerable to undetected tampering. An adversary with file-system access can:

- Modify a log entry to hide evidence of unauthorized activity
- Delete entries entirely to erase a trail
- Reorder entries to obscure the sequence of events

None of these attacks leave visible traces in a conventional log file. The tamper-evident system solves this by making the log entries mathematically dependent on each other, so that altering any one entry breaks the chain and triggers a failed integrity check.

1.2 Objectives

1. Implement cryptographic linkage between consecutive log entries using SHA-256 hashing.
2. Persisting the log chain across the system restarts using a JSON storage backend.
3. Provide a verification function capable of detecting data modification, entry deletion, and entry reordering.
4. Deliver a simple command-line interface for adding entries and verifying integrity.

2. System Design & Architecture

The system is built around two core Python classes: LogEntry and TamperEvidentLog. Together they implement a singly-linked, hash-chained log structure persisted to disk.

2.1 High-Level Architecture

The architecture follows a simple layered model:

Layer	Component	Responsibility
Presentation	main() / CLI Menu	User interaction — add, verify, display, exit
Business Logic	TamperEvidentLog	Manages the chain: load, save, verify, append
Data Model	LogEntry	Represents one record; computes its own hash
Storage	logs.json	Persistent flat-file JSON store
Crypto	hashlib.sha256	Produces 256-bit digest for tamper detection

2.2 The LogEntry Data Structure

Each log entry is a self-describing record containing the following fields:

Field	Type	Description
index	int	Zero-based sequential position in the chain
timestamp	string	ISO-8601 datetime with microsecond precision
event_type	string	Category label — e.g. LOGIN, TRANSACTION, ERROR
description	string	Free-text details of the event
previous_hash	string (hex)	SHA-256 hash of the preceding entry
current_hash	string (hex)	SHA-256 hash computed from all fields above

The current_hash is calculated by concatenating all the other five fields into a single string and computing its SHA-256 digest. This means the hash represents the unique fingerprint of this entry AND its position in the chain — because it encodes the previous entry's hash.

2.3 The Hash Chain Mechanism

The tamper-evident property arises from the structure of the chain. Each entry's current_hash is computed over its own content plus the previous entry's hash. This creates a dependency graph where every entry's validity depends on all entries that precede it.

Chain Formula

H(n) = SHA256(index_n || timestamp_n || event_type_n || description_n || H(n-1))Where || denotes string concatenation and H(0) = SHA256(genesis_data || "0")

This means if entry 3 is altered, its H(3) changes. Entry 4 stores H(3) as its previous_hash — so the stored value now disagrees with the recalculated value. Verification detects the mismatch immediately. The damage propagates forward: every entry after the tampered one fails verification.

2.4 Genesis Entry

The first entry (index 0) is the genesis block. It is created automatically when the system initializes on a fresh log file. Its previous_hash is set to the literal string "0", providing a fixed anchor for the chain. All subsequent entries build upon this foundation.

2.5 Persistence Model

Logs are serialized to a JSON array and written to disk after every new entry. When the system starts, it checks whether logs.json exists. If it does, all entries are deserialized into LogEntry objects and the in-memory list is reconstructed. If not, a genesis entry is created and immediately saved.

3. Tamper Detection Capabilities

The verify_integrity() method iterates through every entry from index 1 onward and performs two independent checks. Together these checks cover all three categories of tampering defined in the task requirements.

3.1 Detection Check 1 — Data Modification

For each entry, the system recomputes the hash from scratch using the stored field values and compares it to the stored current_hash:

```
if current.current_hash != current.calculate_hash():
    return f"Tampering detected at log {i}: data modified"
```

Fig- 1

If even a single character of any field (timestamp, event_type, or description) is altered, calculate_hash() will produce a different value and this condition will be True. The check is O(n) across all fields of the entry and is cryptographically sound — finding a collision in SHA-256 is computationally infeasible with current technology.

3.2 Detection Check 2 — Deletion and Reordering

A second check verifies that the hash chain linkage is intact by comparing each entry's stored previous_hash against the actual hash of the preceding entry:

```
if current.previous_hash != previous.current_hash:
    return f"Tampering detected at log {i}: chain broken (deletion or reordering)"
```

Fig- 2

This catches two distinct attack vectors:

- Deletion — if an entry is removed, the next entry's previous_hash no longer matches its new predecessor's hash.
- Reordering — if entries are shuffled, the previous_hash values point to different entries than the ones now preceding them.

3.3 Tamper Scenario Summary

Attack Type	How Detected	Which Check Fires
Modify a field value	Stored hash mismatches recalculated hash	Check 1 (data modified)
Delete an entry	Next entry's prev_hash doesn't match new predecessor	Check 2 (chain broken)

Reorder entries	prev_hash no longer matches actual predecessor	Check 2 (chain broken)
Insert a forged entry	Chain breaks at insertion point	Check 2 (chain broken)
Silently rebuild chain	Computationally infeasible (SHA-256 preimage resistance)	Both checks

4. Implementation Walkthrough

4.1 Class: LogEntry

LogEntry is a pure data class responsible for holding one record and computing its own fingerprint.

```
class LogEntry:  3 usages
    def __init__(self, index, timestamp, event_type, description, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.event_type = event_type
        self.description = description
        self.previous_hash = previous_hash
        self.current_hash = self.calculate_hash()

    def calculate_hash(self):  2 usages (1 dynamic)
        data = f"{self.index}{self.timestamp}{self.event_type}{self.description}{self.previous_hash}"
        return hashlib.sha256(data.encode()).hexdigest()
```

Fig- 3

The calculate_hash() method is called both at construction time (to set current_hash) and by verify_integrity() at runtime (to validate the stored hash). Using the same method in both contexts guarantees consistency.

4.2 Class: TamperEvidentLog — add_log()

New entries are always appended to the tail of the chain. The method retrieves the last entry's hash to use as the previous_hash for the new entry, ensuring the chain is extended correctly:

```
def add_log(self, event_type, description):  1 usage
    last_log = self.logs[-1]
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")

    entry = LogEntry(
        index=len(self.logs),
        timestamp=timestamp,
        event_type=event_type,
        description=description,
        previous_hash=last_log.current_hash
    )
    self.logs.append(entry)
    self.save()
    print(f"Log {entry.index} added successfully")
```

Fig- 4

4.3 Class: TamperEvidentLog — verify_integrity()

```python
def verify_integrity(self):  1 usage
    for i in range(1, len(self.logs)):
        current = self.logs[i]
        previous = self.logs[i - 1]

        if current.current_hash != current.calculate_hash():
            return f"Tampering detected at log {i}: data modified"

        if current.previous_hash != previous.current_hash:
            return f"Tampering detected at log {i}: chain broken (deletion or reordering)"

    return "All logs are intact"
```

Fig- 5

The function returns immediately upon finding the first violation and reports the exact index at which tampering occurred. If no violation is found across all entries, it confirms the log is intact.

5. Data Flow Diagrams

5.1 Adding a New Log Entry

Step	Actor	Action
1	User (CLI)	Selects option 1, inputs event_type and description
2	TamperEvidentLog	Reads current_hash from self.logs[-1]
3	TamperEvidentLog	Creates LogEntry with previous_hash = last hash
4	LogEntry.__init__	Calls calculate_hash() and stores result as current_hash
5	TamperEvidentLog	Appends entry to self.logs list
6	TamperEvidentLog.save ()	Serializes entire log list to logs.json via json.dump

5.2 Verifying Integrity

Step	Actor	Action
1	User (CLI)	Selects option 2
2	TamperEvidentLog	Iterates i from 1 to len(logs)-1
3	LogEntry	Recalculates hash from stored field values
4	TamperEvidentLog	Compares recalculated hash to stored current_hash
5	TamperEvidentLog	Compares stored previous_hash to actual predecessor's hash
6	TamperEvidentLog	Returns first failure found, or success message

6. Usage Instructions

6.1 Prerequisites

- Python 3.6 or higher (uses f-strings and datetime)
- No external libraries required — uses only Python standard library (hashlib, json, datetime, os)

6.2 Running the System

Save the source code as tamper_log.py and run it from your terminal:

```
python tamper_log.py
```

On first run, a logs.json file is created in the current directory with a genesis entry. On subsequent runs, the existing chain is loaded from disk.

6.3 Menu Options

Option	Action	Example Input
1 — Add Log Entry	Prompts for event type and description, appends to chain	Event Type: LOGINDescription: User admin logged in from 192.168.1.5
2 — Verify Integrity	Runs full chain verification, prints result to console	(no input required)
3 — Display Logs	Prints all entries with all fields to console	(no input required)
4 — Exit	Terminates the program	(no input required)

6.4 Sample Session

```
C:\Users\LOQ\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\LOQ\PyCharm

1. Add Log Entry
2. Verify Log Integrity
3. Display Logs
4. Exit
Enter choice: 1
Event Type: Login
Description: Checking
Log 2 added successfully

1. Add Log Entry
2. Verify Log Integrity
3. Display Logs
4. Exit
Enter choice: 1
Event Type: Transaction
Description: Paind $100
Log 3 added successfully
```

Fig- 6

After altering the data manually in logs.json file.

```
C:\Users\LOQ\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\LOQ\PyCharm

1. Add Log Entry
2. Verify Log Integrity
3. Display Logs
4. Exit
Enter choice: 2
Tampering detected at log 2: data modified
```

Fig- 7

6.5 Understanding the Output File (logs.json)

After adding entries, logs.json will resemble the following structure:

```
[
    {
        "index": 0,
        "timestamp": "2026-03-31 21:49:26.421274",
        "event_type": "GENESIS",
        "description": "System initialized",
        "previous_hash": "0",
        "current_hash": "d37ee24c39aff08673903e2bf7d2d5403a168527fd2ed4775031b29f7fa45511"
    },
    {
        "index": 1,
        "timestamp": "2026-03-31 21:49:37.685019",
        "event_type": "login",
        "description": "1st check",
        "previous_hash": "d37ee24c39aff08673903e2bf7d2d5403a168527fd2ed4775031b29f7fa45511",
        "current_hash": "d14390735a0efc95d4d8671ab6e52856ec78271596cde5c48e606f6ef50fcbda"
    },
    {
        "index": 2,
        "timestamp": "2026-03-31 23:05:27.405535",
        "event_type": "Login",
        "description": "Chesking",
        "previous_hash": "d14390735a0efc95d4d8671ab6e52856ec78271596cde5c48e606f6ef50fcbda",
        "current_hash": "bfb8d588740f02e7884a33e01d70ef66c01ec4c80739fff1c5d33cc375a6ca1e"
    },
    {
        "index": 3,
        "timestamp": "2026-03-31 23:05:59.085291",
        "event_type": "Transaction",
        "description": "Paind $100",
        "previous_hash": "bfb8d588740f02e7884a33e01d70ef66c01ec4c80739fff1c5d33cc375a6ca1e",
        "current_hash": "d7afa8b4a68a6f06155b91ba67620ac59da5671a8c3caa677cee5ae0cbd83045"
```


Fig- 8

7. Design Decisions & Assumptions

7.1 Choice of SHA-256

SHA-256 was chosen for its balance of security and performance. It is a cryptographic hash function from the SHA-2 family, standardized by NIST. Key properties relevant to this application include:

- Determinism — same input always produces the same hash
- Avalanche effect — a single bit change in input produces a completely different output
- Preimage resistance — given a hash H, it is computationally infeasible to find an input M such that SHA256(M) = H
- Collision resistance — it is computationally infeasible to find two inputs M1 and M2 such that SHA256(M1) = SHA256(M2)

7.2 JSON vs. Binary Storage

JSON was selected for human readability and debuggability. In a production system, binary formats or a relational database would be preferred for performance at scale. JSON is appropriate here for an academic demonstration because it allows direct inspection of the chain structure without special tools.

7.3 Microsecond Timestamps

Timestamps include microsecond precision (%f). This reduces — but does not eliminate — the risk of two entries having identical timestamps if added in rapid succession, which could otherwise create ambiguity in audit reconstruction.

7.4 Assumptions and Limitations

Assumption / Limitation	Details
Single-writer model	Concurrent writes from multiple processes can corrupt the chain. A mutex or database transaction would be required for multi-threaded environments.
File system trust boundary	The system detects tampering but cannot prevent it. An attacker with file access can still destroy the file entirely. External append-only storage or WORM media would be needed for prevention.
No digital signatures	The current implementation does not use public-key cryptography. A production system would sign each entry with a private key, allowing independent verification by third parties.
Memory integrity	The in-memory list is not protected. For highest assurance, entries should be verified after every load, not just on user request.

Chain rebuild attack	An attacker who can also recompute the entire chain (rebuild all hashes after modification) could defeat detection. This is addressed in production by combining hash chains with digital signatures and/or external timestamping.

8. Testing & Verification

8.1 Test Cases

Test ID	Scenario	Expected Result	Status
TC-01	Add 3 entries, verify	All logs are intact	PASS
TC-02	Edit description in logs.json, verify	Tampering detected at log N: data modified	PASS
TC-03	Delete middle entry from logs.json, verify	Tampering detected at log N: chain broken	PASS
TC-04	Swap two adjacent entries in logs.json, verify	Tampering detected at log N: chain broken	PASS
TC-05	Delete logs.json, restart system	Genesis entry auto-created, new chain started	PASS
TC-06	Load existing valid log, add entry, verify	All logs are intact	PASS

All test cases passed successfully.

The system correctly detected all three categories of tampering: data modification, entry deletion, and entry reordering. It also handled the edge case of a missing log file by automatically initializing a new chain.

9. Real-World Relevance

Hash-chained audit logs are not merely an academic exercise — they underpin the security architecture of numerous critical systems:

Domain	Application
Blockchain / Cryptocurrency	Every block contains the hash of the previous block, making the entire transaction history tamper-evident (Bitcoin, Ethereum).
Financial Audit Trails	Banks and payment processors maintain hash-chained transaction logs to meet regulatory requirements (PCI-DSS, SOX).
Electronic Health Records	HIPAA-compliant systems use audit logs to track who accessed patient data and when, with integrity guarantees.
Certificate Transparency (CT)	Google's CT logs use Merkle trees (an extension of hash chains) to make TLS certificate issuance publicly auditable.
Cloud Provider Audit Logs	AWS CloudTrail, Azure Monitor, and GCP Audit Logs use cryptographic signing and chaining to guarantee log integrity.

This implementation demonstrates the core principle shared by all these systems: making integrity violation mathematically detectable rather than relying on access control alone.

10. Conclusion

This report has presented a complete Tamper-Evident Logging System that fulfills all specified functional requirements. The implementation leverages SHA-256 cryptographic hashing to create a blockchain-inspired chain of log entries where each record is cryptographically bound to its predecessor.

The key achievements of this implementation are:

Requirement	Implementation	Result
Detect data modification	Check 1 in verify_integrity(): recalculate hash, compare to stored	Achieved
Detect entry deletion	Check 2 in verify_integrity(): compare prev_hash to actual predecessor	Achieved
Detect entry reordering	Check 2 in verify_integrity(): same mechanism detects broken links	Achieved
Pinpoint tampering location	Returns index of first violated entry	Achieved
Persistent storage	JSON serialization with save/load cycle	Achieved
Simple interface	4-option CLI menu	Achieved

While the current implementation is designed for educational purposes, the underlying cryptographic principle is production-grade. Extending it with digital signatures, an append-only storage backend, and concurrent write support would produce a system suitable for real audit logging scenarios.